PROJECT REPORT

ASCII Art Animation Player

A menu-driven, terminal-based ASCII animation program built in Python

Submitted by: __Dhanvantari_Saini_____

Registration Number: _____26BCE10895_____

Course: B.Tech in CSE Core
Subject: Introduction to Problem Solving and Programming
Course Code: CSE1021
Faculty: Dr. Siddharth Singh Chouhan

Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Target Users
6. Functional Requirements
7. Non-Functional Requirements
8. Technology and Tools Used
9. Project Structure
10. System Architecture
11. Program Work Flow
12. Flowchart
13. UML Diagrams
    - 13.1 Use Case Diagram
    - 13.2 Sequence Diagram
    - 13.3 Component Diagram
14. Design Decisions
15. Implementation
16. Grading Criteria
17. Input Validation
18. Testing
19. Sample Output
20. Challenges Faced
21. Learning Outcomes
22. Future Enhancements
23. Conclusion
24. References

1. Introduction

Terminal applications remain one of the simplest ways to understand how a program manages state, user input, and timing without the added complexity of a graphical toolkit. This project, the ASCII Art Animation Player, is a menu-driven Python program that plays a series of short animations built entirely out of text characters inside a standard console window. It was developed as part of the Introduction to Problem Solving and Programming coursework, with the goal of applying core programming constructs — loops, conditionals, functions, lists, and dictionaries — to a single, cohesive project rather than isolated exercises.

The program repeatedly clears the screen, prints a new frame of ASCII art, and pauses briefly, which is enough to create a convincing illusion of motion using nothing but the Python standard library.

2. Problem Statement

Most beginner programming exercises are short and disconnected, which makes it hard to practice combining control flow, data structures, and user interaction in one place. There is a need for a single, self-contained program that a student can build incrementally — starting from a basic menu, then adding one animation at a time — while reinforcing fundamental concepts such as input validation, loop control, and the use of lists and dictionaries to store structured data. The program must also run on any standard computer without installing extra graphics libraries, so it needs to work using only text output in a terminal.

3. Objectives

- Build a menu-driven console program that lets the user pick from multiple animations.
- Simulate motion in a text-only terminal using a clear-print-sleep loop.
- Practice storing and iterating over structured data using lists and dictionaries.
- Use the random module to create procedural, non-repeating effects (e.g. wind, rain, fireworks).
- Apply input validation so the program never crashes on unexpected user input.
- Write modular, well-commented code with one function per animation.

4. Scope of the Project

The project covers the design and implementation of a Python console application containing a main menu loop and twelve independent animation functions, organized into clearly separated modules. It is limited to text-based rendering inside a terminal window of roughly 80 columns by 24 rows and does not include sound, color libraries, or any graphical windowing system. The scope also includes basic input validation, graceful handling of keyboard interrupts, and clear code comments explaining the animation and rendering logic. It does not extend to networked features, saving/loading animation state, or a graphical user interface — these are noted as possible future extensions.

5. Target Users

- Beginner programming students learning loops, functions, and basic data structures.
- Anyone who wants a lightweight, dependency-free terminal screensaver or demo.
- Instructors looking for a compact example that touches many core Python concepts.

6. Functional Requirements

- The program shall display a numbered menu of all available animations plus a Quit option.
- The program shall accept a numeric choice from the user and run the corresponding animation.
- The program shall reject non-numeric or out-of-range input and re-prompt the user.
- Each animation shall run as a self-contained loop that clears and redraws the screen.
- The program shall return to the main menu automatically once an animation finishes.
- The program shall allow the user to interrupt a running animation (Ctrl+C) and return to the menu.
- The program shall exit cleanly when the user selects the Quit option.

7. Non-Functional Requirements

- Portability: the program should run unmodified on both Windows and Unix-like systems.
- No external dependencies: only Python's standard library may be used.
- Readability: code should be commented clearly enough for a beginner to follow.
- Performance: each animation frame should render quickly enough to appear smooth (no long pauses).
- Robustness: invalid input or an interrupted animation must never crash the program.
- Compact display: all ASCII art should fit within an 80x24 terminal window.

8. Technology and Tools Used

- Python 3 — the sole programming language used for the project.
- Standard library modules only: os (screen clearing), time (frame delays), random (procedural effects), sys (clean exit).
- Visual Studio Code — used as the code editor during development.
- Git and GitHub — used for version control and to host/share the project repository.
- A standard OS terminal / command prompt — used to run and test the program.

9. Project Structure

The project is organized into several focused Python modules rather than one long script, which keeps each part of the program easy to read, test, and extend:

- main.py — the program's entry point; contains the while-loop that prints the menu, validates input, and calls the chosen animation.
- renderer.py — screen helper functions: clear_screen(), render_frame(), wait_for_enter(), and the run_animation() wrapper that catches Ctrl+C.
- animations_basic.py — Blinking Eyes, Swimming Fish, Grass in the Wind.
- animations_motion.py — Bouncing Ball, Bouncing Text, Loading Spinner.
- animations_random.py — Rain Falling, Fireworks, Starfield.
- animations_misc.py — Ticking Clock, Countdown Timer, Flickering Candle.
- README.md and statement.md — project documentation files required for the GitHub submission.

10. System Architecture

The architecture is intentionally simple and linear, reflecting the procedural style expected at this stage of the course. The user interacts only with the terminal; the Main Menu Controller reads input and hands control to one of the Animation Functions, which supply frame data (either hard-coded ASCII strings or grids built with the random module) to a single shared Rendering Engine. The rendering engine is the only part of the program that talks to the operating system to clear the screen and pause execution, which keeps that logic in one place instead of repeating it in every animation.

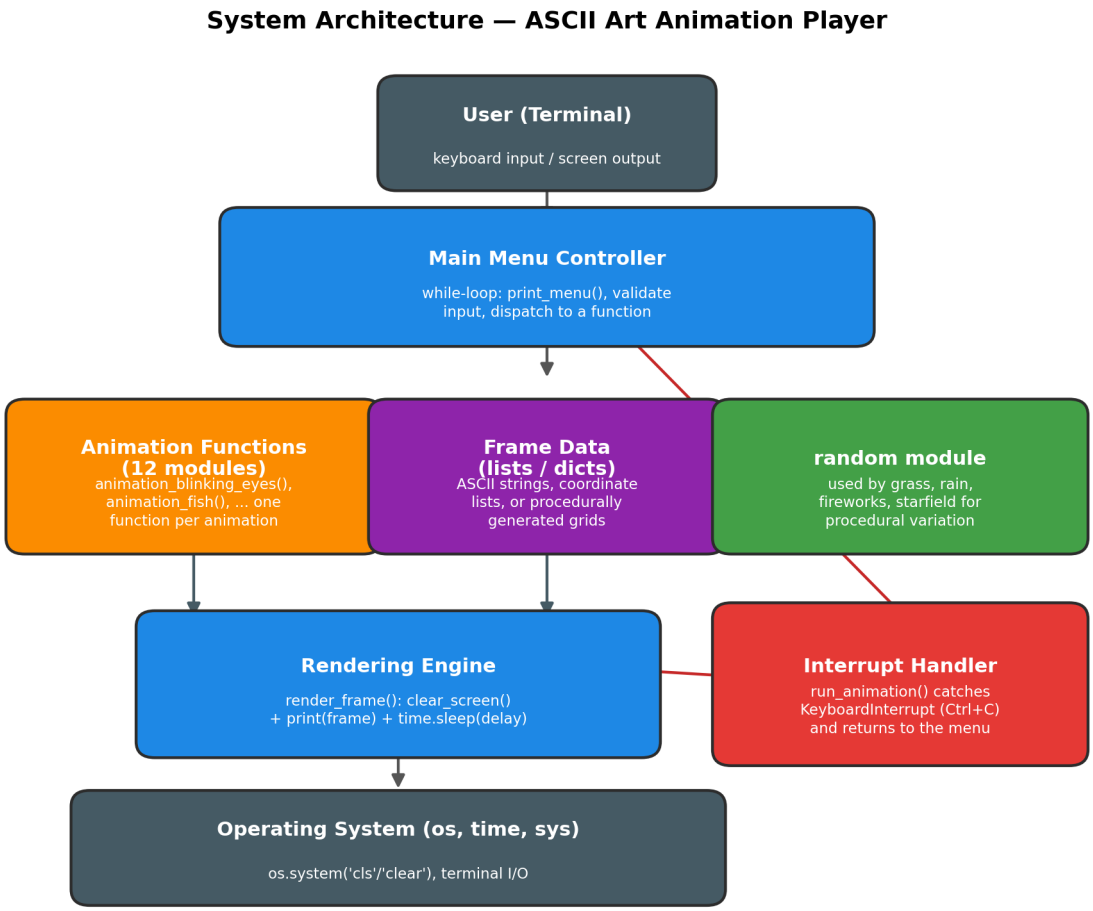


Figure 1: System architecture of the ASCII Art Animation Player.

11. Program Work Flow

At a high level, the program starts, shows the menu, and waits for input. Invalid input is rejected with a short message before the menu is shown again. A valid choice of 0 ends the program; any other valid choice calls the matching animation function, which loops through its frames (clearing the screen, drawing the next frame, and sleeping briefly) until it finishes naturally or the user presses Ctrl+C. Either way, control returns to the main menu so the user can pick another animation or quit.

12. Flowchart

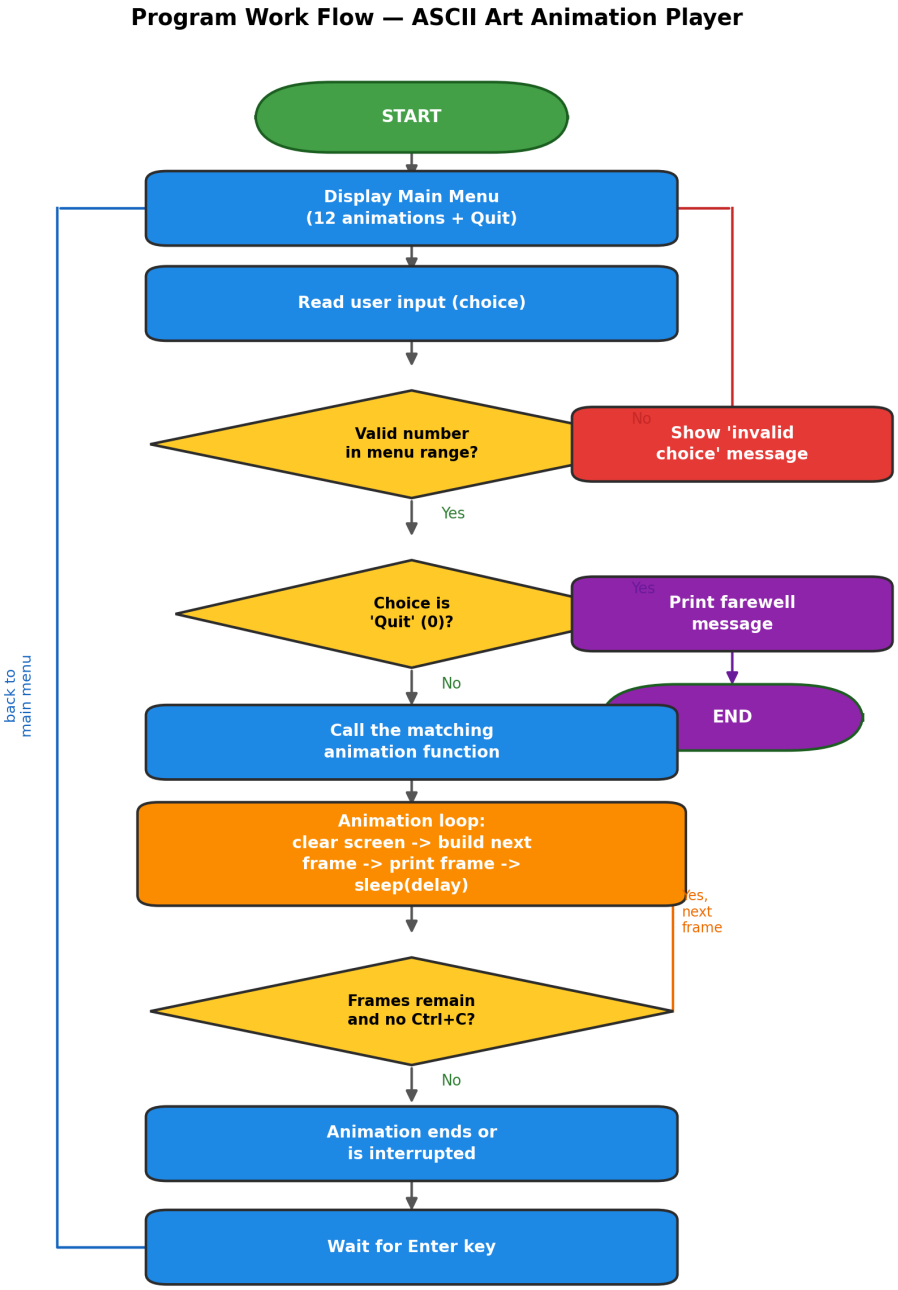


Figure 2: Program flow from startup to exit.

13. UML Diagrams

The following diagrams describe the system from three complementary UML perspectives: what the user can do with it, how objects collaborate at runtime, and how the code is organized into components.

13.1 Use Case Diagram

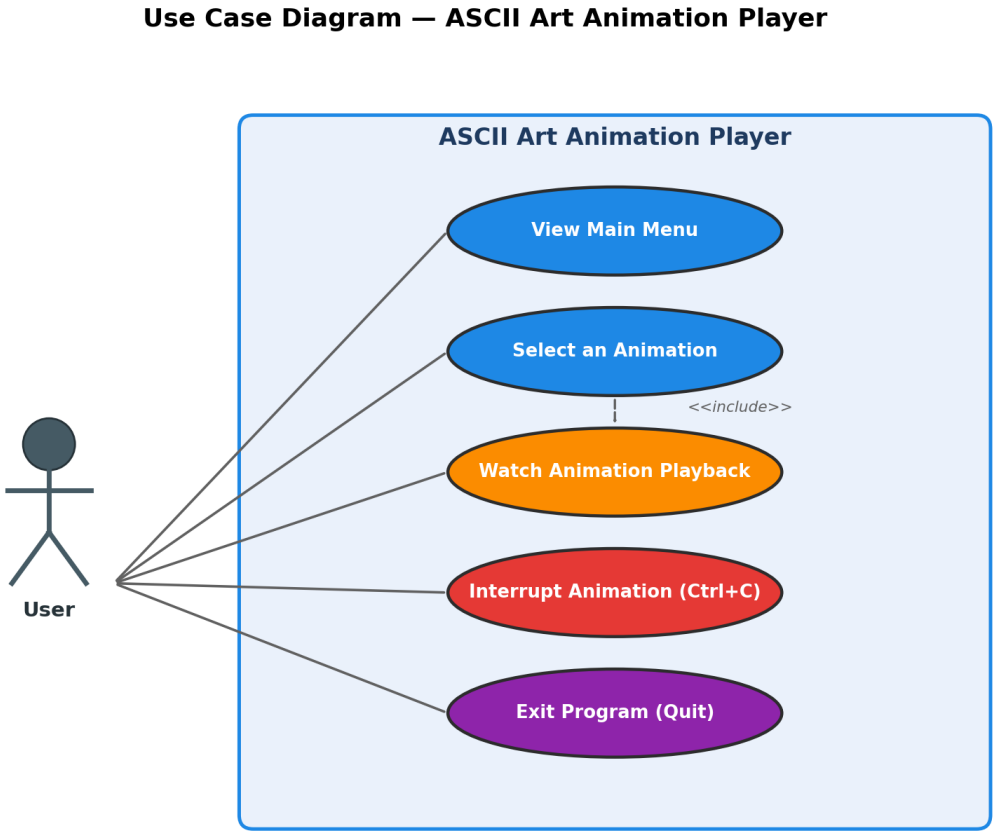


Figure 3: Use case diagram — the single actor (User) and the actions available to them.

13.2 Sequence Diagram

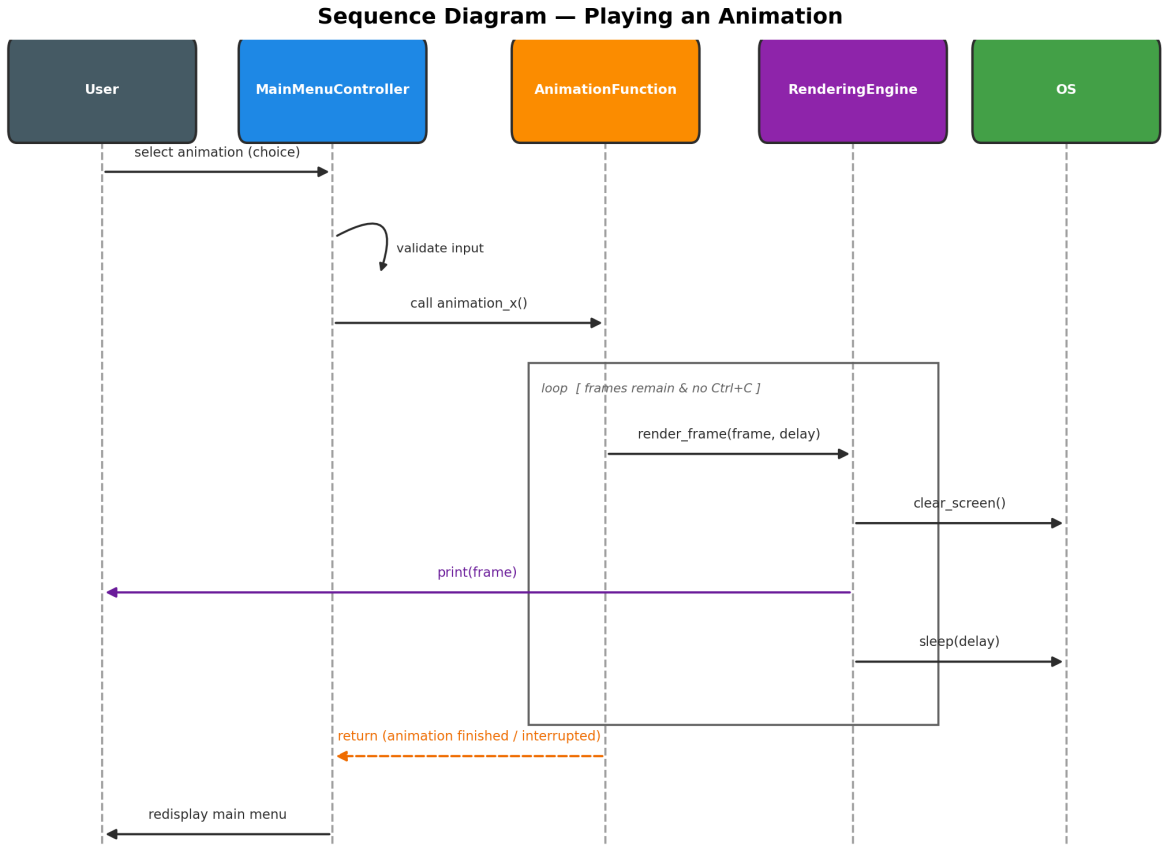


Figure 4: Sequence diagram showing the messages exchanged while one animation plays.

13.3 Component Diagram

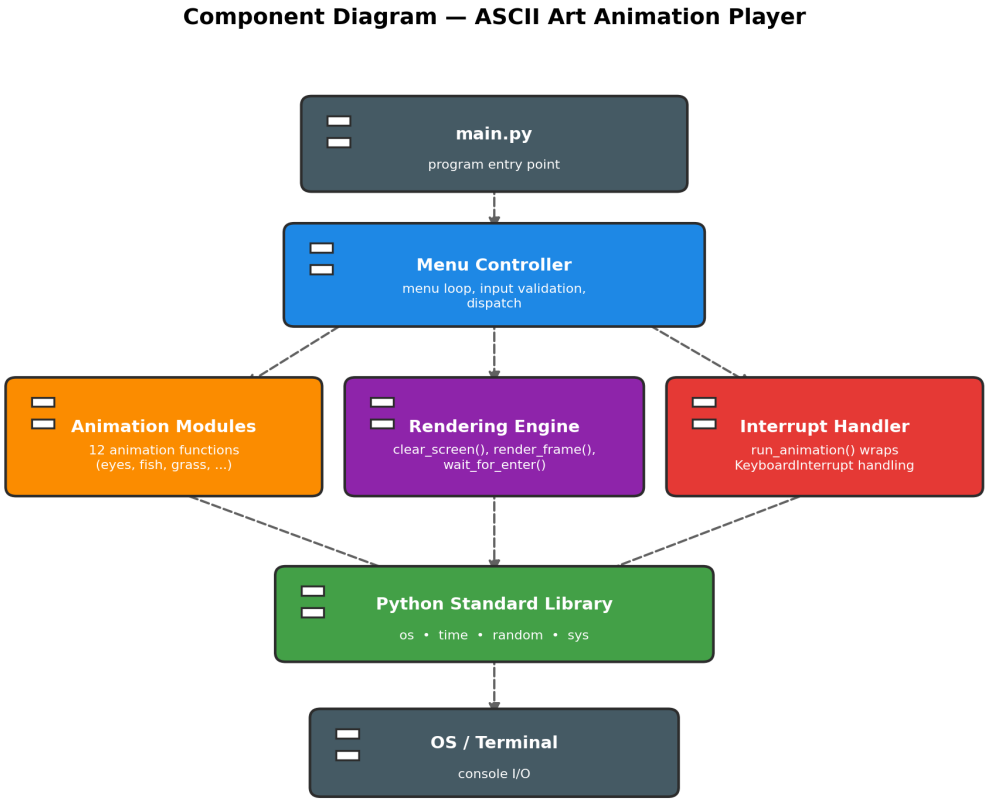


Figure 5: Component diagram showing how the modules depend on one another.

14. Design Decisions

- One function per animation: keeps each effect self-contained, easy to test, and easy to extend with new animations later.
- A single shared render_frame() helper: avoids repeating the clear-print-sleep pattern in every animation.
- Animations are grouped into a small number of modules by theme (basic/motion/random/misc) rather than one file per animation, keeping the folder structure manageable while still meeting the project's modularity requirement.
- random is used for gradual, biased changes (e.g. a 'gust direction' for the grass) rather than pure per-character randomness, so effects look like natural motion instead of static noise.
- KeyboardInterrupt is caught in one central wrapper (run_animation) instead of inside every animation, so Ctrl+C always behaves the same way.
- Menu options and their functions are paired in one list (MENU_OPTIONS), so adding a new animation only requires adding one line.

15. Implementation

The program is implemented in pure Python using only the os, time, random, and sys modules. The main loop in main() prints the menu built from MENU_OPTIONS, reads the user's choice with input(), and validates it

with str.isdigit() before converting it to an integer. Each animation function builds its frames either as literal multi-line strings (Blinking Eyes, Countdown), by adjusting string padding each frame (Swimming Fish, Bouncing Ball/Text), or by generating a 2D grid of characters that is redrawn every frame (Grass in the Wind, Rain, Fireworks, Starfield). Every animation is called through run_animation(), which wraps it in a try/except block so that pressing Ctrl+C stops the animation and returns to the menu instead of terminating the whole program.

16. Grading Criteria

The following rubric outline can be used to evaluate the project; marks may be adjusted to match the exact weightage given by the faculty.

Criteria	Description	Marks
Correctness	Program runs without errors and all menu options work as expected	
Code Quality	Readable, modular, and clearly commented code	
Use of Core Concepts	Appropriate use of loops, conditionals, functions, lists/dictionaries	
Input Validation	Program handles invalid or unexpected input gracefully	
Creativity	Originality and variety among the implemented animations	
Documentation	Quality and completeness of this project report	
Viva / Presentation	Ability to explain the code and design choices	

17. Input Validation

- Menu input is checked with str.isdigit() before conversion, so letters or symbols never crash the program.
- Numeric input outside the valid menu range shows an error message and redisplays the menu.
- Ctrl+C during an animation is caught with a try/except KeyboardInterrupt block, returning cleanly to the menu.
- Ctrl+C at the main menu itself is also caught so the program always exits with a clean message.

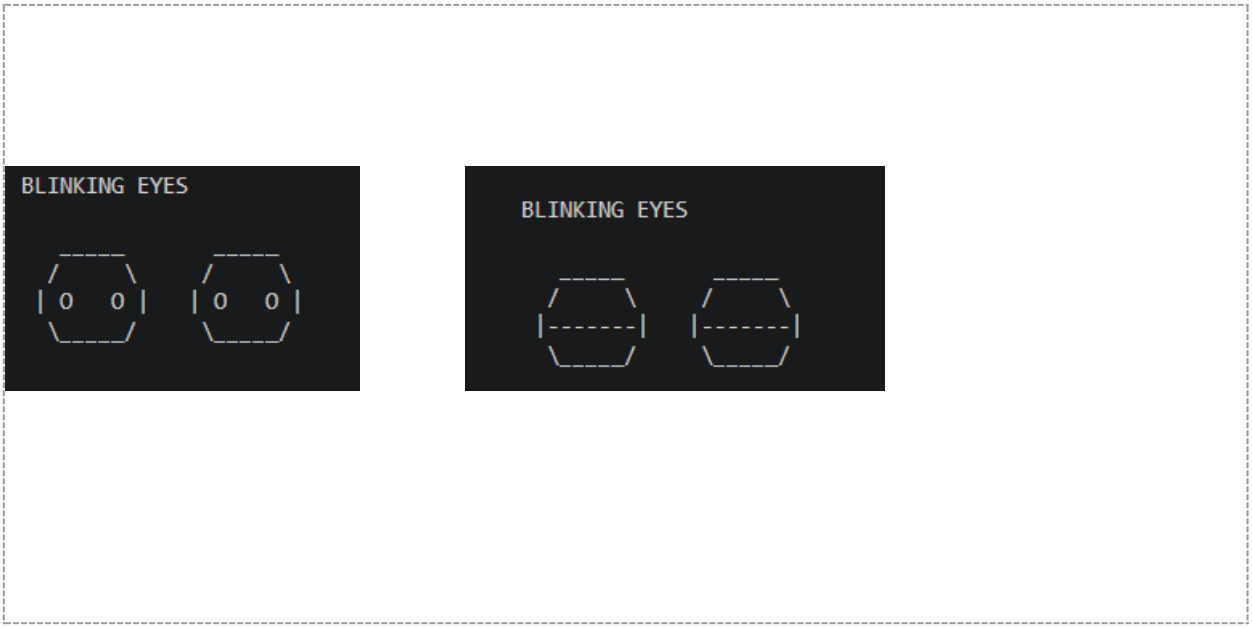
18. Testing

Testing was carried out manually by running the program and exercising each menu path. The table below summarizes the main test cases.

Test Case	Input	Expected Result	Result
Valid animation choice	e.g. 1-12	Selected animation plays, then returns to menu	Pass
Quit option	0	Program prints a farewell message and exits	Pass
Non-numeric input	e.g. 'abc'	Error message shown, menu redisplayed	Pass
Out-of-range number	e.g. 99	Error message shown, menu redisplayed	Pass
Interrupt during animation	Ctrl+C	Animation stops, control returns to menu	Pass

19. Sample Output

A short description of what is shown, followed by a terminal screenshot pasted below.



20. Challenges Faced

- Making the screen-clear command work identically on both Windows and Unix-like systems.
- Tuning sleep() delays so animations looked smooth without flickering or running too fast to see.
- Keeping every animation's ASCII art within an 80x24 terminal so nothing wraps or scrolls awkwardly.
- Making the random-based effects (grass, rain, fireworks) look like natural motion rather than random noise.
- Ensuring Ctrl+C during any animation returns to the menu instead of closing the whole program.

21. Learning Outcomes

- Gained practical experience combining loops, conditionals, and functions in one larger program.
- Learned how to represent and iterate over animation data using lists and dictionaries.
- Understood how the random module can be used for controlled, natural-looking variation.
- Practiced structuring a program into small, reusable modules instead of one long script.
- Learned to anticipate and handle invalid input and user interrupts gracefully.
- Got hands-on practice with Git/GitHub for tracking and sharing the project.

22. Future Enhancements

- Add color support using a lightweight library such as colorama for richer visuals.
- Allow users to control animation speed or duration from the menu.
- Add a configuration file so users can enable/disable specific animations.
- Add automated unit tests (e.g. with pytest) for the input-validation logic.
- Package the project so it can be installed and run as a command-line tool (pip install).

- Port the renderer to a curses-based UI for flicker-free, non-blocking animation.

23. Conclusion

The ASCII Art Animation Player successfully demonstrates how fundamental programming concepts — loops, conditionals, functions, lists, and dictionaries — can be combined into a single interactive program using nothing beyond Python's standard library. The menu-driven structure, modular animation functions, and centralized rendering and error-handling logic make the project easy to extend with additional animations in the future. Overall, the project met its objective of reinforcing core problem-solving and programming skills through a practical, visual, and enjoyable outcome.

24. References

- Python Official Documentation — Python programming concepts and built-in functions.
- Visual Studio Code Documentation — Code editing and development environment.
- Git Documentation — Version control and project management.
- GitHub Documentation — Repository management and project documentation.
- Course/Faculty Project Guidelines — Requirements and evaluation criteria provided for the project.
- Project development was based mainly on Python programming concepts taught in the course.